

Department of Electrical and Computer Engineering
Queen's University
ELEC 374 Digital Systems Engineering
Laboratory Project

Winter 2026

Designing a Simple RISC Computer: CPU Specification

1. Objectives

The purpose of this project is to design, simulate, implement, and verify a Simple RISC Computer (Mini SRC), consisting of a simple RISC processor, memory, and I/O. You are to use the Intel/Altera Quartus Prime Design Software and ModelSim-FPGA for this purpose, and your design is to be implemented on the Cyclone V chip (5CEBA4F23C7) of the [DE0-CV development board](#).

The Mini SRC is similar in concept to the original SRC described in the [Lab Reader](#), reproduced from the text by Heuring and Jordan. The Datapath, Control Unit, and Memory Interface for Mini SRC have the same relationship as shown in Figure 4.1 on page 142 of the Lab Reader for the SRC system. The processor design is similar in concept to the information presented in figures and tables on pages 143 through 167 of the Lab Reader. As part of the learning process for the CPU design project, make sure you carefully read the Mini SRC CPU Specification (this document) and the descriptions of different lab phases, the [CPU Design Project Tutorial](#), the [Introduction to Intel/Altera Quartus Prime Design Software, ModelSim-FPGA Starter Edition, and DE0-CV Development Board](#), and the Lecture Slides on Computer Arithmetic, and Verilog or VHDL.

2. Processor Specification

The Mini SRC is a 32-bit machine, having a 32-bit datapath and sixteen 32-bit registers R0 to R15, with R0 to R7 as general-purpose registers, R8 to R11 as the four argument registers, R12 as the return address register (RA) holding the return address for a *jal* instruction, R13 and R14 as the two return value registers, and R15 as the stack pointer (SP). It also has two dedicated 32-bit registers HI and LO for multiplication and division instructions. Note that Mini SRC does not have a condition code register. Rather, it allows any of the general-purpose registers to hold a value to be tested for conditional branching. The memory unit is 512 words. The following is a formal definition of the Mini SRC.

Processor State

PC<31..0>:	32-bit Program Counter (PC)
IR<31..0>:	32-bit Instruction Register (IR)
R[0..15]<31..0>:	16 32-bit registers, named R[0] through R[15]
R[0..7]<31..0>:	Eight General-Purpose Registers
R[8..11]<31..0>:	Four Argument Registers
R[12]<31..0>:	Return Address Register (RA)
R[13..14]<31..0>:	Two Return Value Registers
R[15]<31..0>:	Stack Pointer (SP)
HI<31..0>:	32-bit HI Register dedicated to keep the high-order word of a Multiplication product, or the Remainder of a Division operation
LO<31..0>:	32-bit LO Register dedicated to keep the low-order word of a Multiplication product, or the Quotient of a Division operation

Memory State

Mem[0..511]<31..0>:	512 words (32 bits per word) of memory
MDR<31..0>:	32-bit Memory Data Register
MAR<31..0>:	32-bit Memory Address Register

I/O State

In.Port<31..0>:	32-bit Input Port
Out.Port<31..0>:	32-bit Output Port
Run.Out:	Run/halt Indicator
Stop.In:	Stop signal
Reset.In:	Reset signal

The *Arithmetic Logic Unit* (ALU) performs 13 operations: addition, subtraction, multiplication, division, shift right, shift right arithmetic, shift left, rotate right, rotate left, logical AND, logical OR, Negate (2's complement), and NOT (1's complement).

The instructions in Mini SRC are one-word (32-bit) long each. They can be categorized as **Load and Store** instructions, **Arithmetic and Logical** instructions, **Conditional Branch** and **Jump** instructions, **Input/Output** instructions, and **miscellaneous** instructions. There are no push and pop instructions (they can be implemented by other instructions). The following six addressing modes are supported: **Direct**, **Indexed**, **Register**, **Register Indirect**, **Immediate**, and **Relative**.

2.1 Instruction Formats

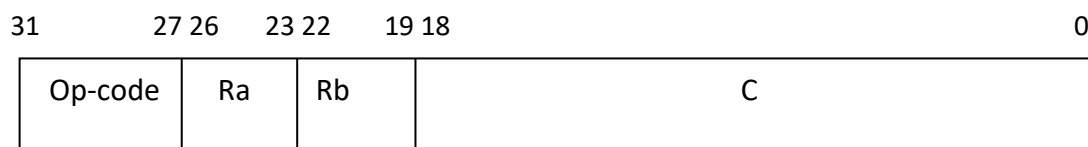
There are five instruction formats, as shown in the table below.

Name	Fields					Comments
Field size	31..27 5 bits	26..23 4 bits	22..19 4 bits	18..15 4 bits	14..0 15 bits	All instructions are 32-bit long
R-Format	OP-code	Ra	Rb	Rc	Unused	Arithmetic/Logical
I-Format	OP-code	Ra	Rb	Constant C / Unused		Arithmetic/Logical; Load/Store; Imm.
B-Format	OP-code	Ra	C2	Constant C		Branch
J-Format	OP-code	Ra	Unused			Jump; Input/Output; Special
M-Format	OP-code	Unused				Misc.

The instruction formats for different categories are detailed below:

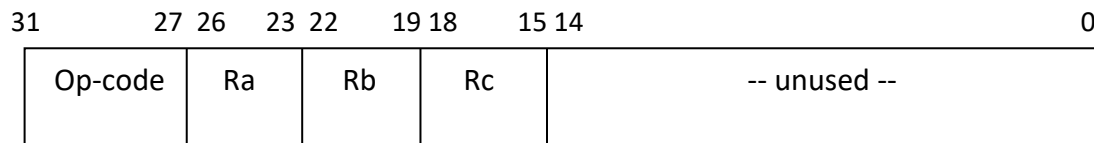
- **Load and Store** instructions: operands in memory can be accessed only through load/store instructions.

(a) **ld, ldi, st** I-Format



- Arithmetic and Logical instructions:

(a) add, sub, and, or, shr, shra, shl, ror, rol R-Format



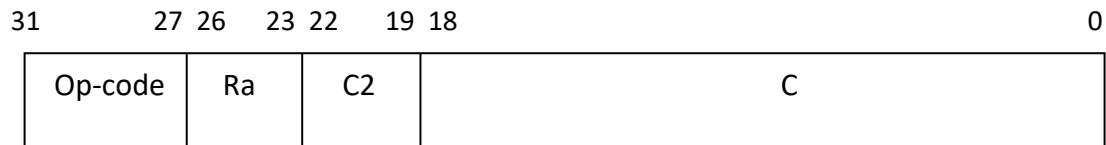
(b) addi, andi, ori I-Format



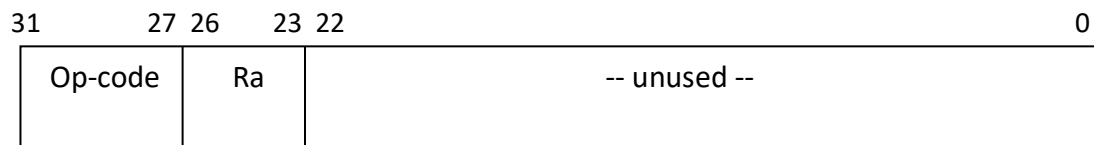
(c) mul, div, neg, not I-Format



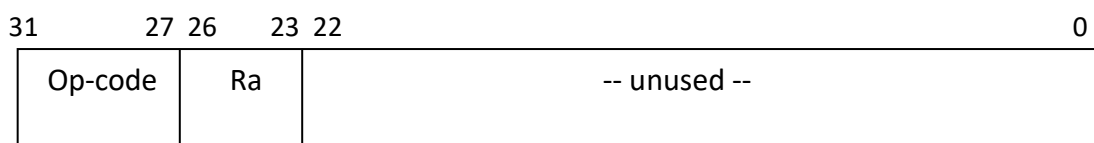
- Branch instructions: brzr, brnz, brmi, brpl B-Format



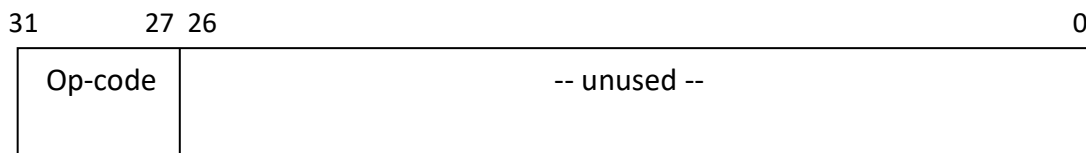
- Jump instructions: jr, jal J-Format



- Input/Output and MFHI/MFLO instructions: in, out, mfhi, mflo J-Format



- Miscellaneous instructions: *nop*, *halt* M-Format



Op-code: specifies the operation to be performed.

Ra, Rb, Rc: 0000: R0, 0001: R1, ..., 1111: R15

C: constant (data or address)

C2: condition --00: branch if zero
 --01: branch if nonzero
 --10: branch if positive
 --11: branch if negative

Notation: x: 0 or 1
 - : unused

2.2 Instructions

The instructions (with their op-code patterns shown in parentheses) perform the following operations:

Arithmetic and Logical Instructions

(a): *add*, *sub*, *and*, *or*, *shr*, *shra*, *shl*, *ror*, *rol*

add: Add (00000xxxxxxxxxxxxx-----)	$R[Ra] \leftarrow R[Rb] + R[Rc]$	add Ra, Rb, Rc
sub: Sub (00001xxxxxxxxxxxxx-----)	$R[Ra] \leftarrow R[Rb] - R[Rc]$	sub Ra, Rb, Rc
and: AND (00010xxxxxxxxxxxxx-----)	$R[Ra] \leftarrow R[Rb] \wedge R[Rc]$	and Ra, Rb, Rc
or: OR (00011xxxxxxxxxxxxx-----)	$R[Ra] \leftarrow R[Rb] \vee R[Rc]$	or Ra, Rb, Rc
shr: Shift Right (00100xxxxxxxxxxxxx-----)	Shift right R[Rb] into R[Ra] by count in R[Rc]	shr Ra, Rb, Rc
shra: Shift Right Arithmetic (00101xxxxxxxxxxxxx-----)	Shift right arithmetic R[Rb] into R[Ra] by count in R[Rc]	shra Ra, Rb, Rc
shl: Shift Left (00110xxxxxxxxxxxxx-----)	Shift left R[Rb] into R[Ra] by count in R[Rc]	shl Ra, Rb, Rc

ror: Rotate Right (00111xxxxxxxxxxxxx-----)	Rotate right R[Rb] into R[Ra] by count in R[Rc]	ror	Ra, Rb, Rc
rol: Rotate Left (01000xxxxxxxxxxxxx-----)	Rotate left R[Rb] into R[Ra] by count in R[Rc]	rol	Ra, Rb, Rc

(b): addi, andi, ori

addi: Add Immediate (01001xxxxxxxxxxxxxxxxxxxxxxxxxxxxx)	$R[Ra] \leftarrow R[Rb] + C \text{ (sign-extended)}$ Immediate addressing If $C = 0 \rightarrow$ instruction acts like a simple register transfer If $C \neq 0$ and $Ra = Rb \rightarrow$ Increment/decrement instruction Similar to Ldi, however Rb can be any register	addi	Ra, Rb, C
andi: AND Immediate (01010xxxxxxxxxxxxxxxxxxxxxxxxxxxxx)	$R[Ra] \leftarrow R[Rb] \wedge C \text{ (sign-extended)}$ Immediate addressing	andi	Ra, Rb, C
ori: OR Immediate (01011xxxxxxxxxxxxxxxxxxxxxxxxxxxxx)	$R[Ra] \leftarrow R[Rb] \vee C \text{ (sign-extended)}$ Immediate addressing	ori	Ra, Rb, C

(c): div, mul, neg, not

div: Divide (01100xxxxxxxx-----)	$HI, LO \leftarrow R[Ra] \div R[Rb]$	div	Ra, Rb
mul: Multiply (01101xxxxxxxx-----)	$HI, LO \leftarrow R[Ra] \times R[Rb]$	mul	Ra, Rb
neg: Negate (01110xxxxxxxx-----)	$R[Ra] \leftarrow -R[Rb]$	neg	Ra, Rb
not: NOT (01111xxxxxxxx-----)	$R[Ra] \leftarrow \overline{R[Rb]}$	not	Ra, Rb

Load and Store Instructions

ld, ldi, st:

ld: Load Direct (10000xxxx0000xxxxxxxxxxxxxxxxxxxxx)	$R[Ra] \leftarrow M[C \text{ (sign-extended)}]$ Direct addressing, Rb = R0
--	---

Assembly language

ld Ra, C

ld: Load Indexed/Register Indirect
(10000xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx)

$R[Ra] \leftarrow M[R[Rb] + C \text{ (sign-extended)}]$ ld Ra, C(Rb)
Indexed addressing, Rb $\neq$ R0
If C = 0 $\rightarrow$ Register Indirect addressing

ldi: Load Immediate
(10001xxxx0000xxxxxxxxxxxxxxxxxxxxxx)

$R[Ra] \leftarrow C \text{ (sign-extended)}$ ldi Ra, C
Immediate addressing, Rb = R0

(10001xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx)

$R[Ra] \leftarrow R[Rb] + C \text{ (sign-extended)}$ ldi Ra, C(Rb)
Immediate addressing, Rb $\neq$ R0
If C = 0 $\rightarrow$ instruction acts like a simple register transfer
If C $\neq$ 0 and Ra = Rb $\rightarrow$ Increment/decrement instruction

st: Store Direct
(10010xxxx0000xxxxxxxxxxxxxxxxxxxxxx)

$M[C \text{ (sign-extended)}] \leftarrow R[Ra]$ st C, Ra
Direct addressing, Rb = R0

st: Store Indexed/Register Indirect
(10010xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx)

$M[R[Rb] + C \text{ (sign-extended)}] \leftarrow R[Ra]$ st C(Rb), Ra
Indexed addressing, Rb $\neq$ R0
If C = 0 $\rightarrow$ Register Indirect addressing

Jump Instructions

jal, jr

jal: jump and link
(10011xxxx-----)

$R[8] \leftarrow PC + 1$ jal Ra
 $PC \leftarrow R[Ra]$

jr: return from procedure
(10100xxxx-----)

$PC \leftarrow R[Ra]$ jr Ra
If Ra = R8, it is for procedure return

Conditional Branch Instructions

brzr, brnz, brmi, brpl

Branch
(10101xxxx--xxxxxxxxxxxxxxxxxxxxxx)

$PC \leftarrow PC + 1 + C \text{ (sign-extended)}$ if R[Ra] meets the condition

Condition: --00: branch if zero	brzr	Ra, C
--01: branch if nonzero	brnz	Ra, C
--10: branch if positive	brpl	Ra, C
--11: branch if negative	brmi	Ra, C

Input/Output and MFHI/MFLO Instructions

in, out, mflo, mfhi

in: Input (10110xxxx-----)	R[Ra] ← In.Port	in	Ra
out: Output (10111xxxx-----)	Out.Port ← R[Ra]	out	Ra
mfhi: Move from HI (11000xxxx-----)	R[Ra] ← HI	mfhi	Ra
mflo: Move from LO (11001xxxx-----)	R[Ra] ← LO	mflo	Ra

Miscellaneous Instructions

nop, halt

nop: No-operation (11010-----)	Do nothing	nop
halt: Halt (11011-----)	Halt the control stepping process	halt

3. Design Phases and Final Report

There are **four design phases** in this group project. Labs start in the third week of classes, and you are **strongly advised to start working on your project early** and follow the guidelines in each phase. **Only three students** are allowed per group. You may use an all HDL (Verilog or VHDL) design approach for your CPU. A Schematic, mixed HDL/Schematic, or a mixed Verilog/VHDL design methodology is not recommended due to the scope of project.

You are to design your project in Quartus and simulate in ModelSim. Follow the guidelines and design steps in the [CPU Design Project Tutorial](#) for each phase. Also, you are **highly advised to consult Section 6, "A Simple Datapath Design in Verilog"**, of the [Introduction to Intel/Altera Quartus Prime Design Software, ModelSim-FPGA Starter Edition, and DE0-CV Development Board](#) tutorial document, and the associated HDL files, as a starting point for designing your registers, bus, data transfer over the bus, testbenches, etc. Also, refer to the [ModelSim](#) tutorial. There are also examples in the Lecture Slides on Verilog and VHDL that can help with your design, simulation, and implementation of your lab project.

Phase 1: The processor Datapath will be partially designed and tested using Functional Simulation in Phase 1. Phase one is worth 7% of the course mark. You will demo your design and simulation results in the lab, and then upload your lab report to onQ, consisting of HDL code, schematic screenshots (if any), testbenches, and

Functional Simulation results by 11:59pm on the day of demo. Consult the CPU Phase 1 document and the CPU Phase 1 Demo, Report Submission, and Marking Guidelines.

Phase 2: The Datapath will be complemented by adding the “Select and Encode logic”, “CON FF Logic”, branch and jump instructions, “Memory Subsystem”, and the “Input/Output Ports”. It will be tested using Functional Simulation. Phase two is worth 7% of the course mark. You will demo your design and simulation results in the lab, and then upload your lab report to onQ, consisting of HDL code, schematic screenshots (if any), testbenches, Functional Simulation results, and contents of the memory before and after execution of load and store instructions by 11:59pm on the day of demo. Consult the CPU Phase 2 document and the CPU Phase 2 Demo, Report Submission, and Marking Guidelines.

Phase 3: The Control Unit will be designed in Verilog or VHDL, and tested using Functional Simulation. You will be provided with a test program to verify your Control Unit. Phase three is worth 5% of the course mark. You will demo your design and simulation results in the lab, and then upload your lab report to onQ, consisting of HDL code, schematic screenshots (if any), contents of the memory before and after the program run, and Functional Simulation results by 11:59pm on the day of demo. Consult the CPU Phase 3 document and the CPU Phase 3 Demo, Report Submission, and Marking Guidelines.

Phase 4: The Datapath and Control Unit will be tested together using both Functional Simulation and implementation on the DE0-CV FPGA development board. You will be provided with a test program to verify your CPU design and implementation. Phase four is worth 3% of the course mark. You will demo your design, simulation, and on-board FPGA results in the lab. There is NO lab report submission for this phase. Consult the CPU Phase 4 document and CPU Phase 4 Demo and Marking Guidelines.

Lab Final Report: You will submit a comprehensive lab final report to onQ. The objective of the Lab Final Report is to provide the students with an opportunity to present their work, discuss their achieved goals, and plan for future work. The Lab Final Report is NOT a lab report for Phase 4; it is a lab report for the entire project. You will describe all aspects of your CPU design project, project specification, project design and implementation, evaluation results and analysis, discussions, conclusion and future work, and the required appendices. The appendices include your final HDL code, schematic screenshots (if any), Functional Simulation results, contents of the memory before and after the program run, and screenshots of the FPGA board showing the displays and switches when running your program. The lab final report is worth 3% of the course mark. Consult the Lab Final Report Guidelines and the Lab Final Report Marking Guidelines.

4. Bonus Mark

There will be up to 5% **bonus marks** if you design, simulate, and implement, for instance, a 3-bus (or more) architecture, advanced techniques that we learned in class for ALU operations, interrupt/exception handling, stack, new instructions, as well as any other advanced techniques such as pipelining, branch prediction, hazard detection, and superscalar design, etc., to improve performance. You are **advised to tackle the 1-bus architecture first**, and when you feel comfortable with your design, then aim for any other improvements.

5. Lab Schedule and General Guidelines

For complete lab schedule and deadlines for each phase and the final lab report, consult the [ELEC 374 Labs page](#) and [ELEC 374 Course Schedule](#).

- Students must attend and actively participate in the design process in the labs with their group.
- Lab demos will be done during the labs, and all members of lab groups must be present during their lab demo. The TAs will ask each member of the lab groups questions about their design and implementation, and contributions.
- Multiple (partial) lab demos for the same phase are NOT allowed, due to the large number of groups in the labs. When you are ready to demo your design, that would be your final demo for that phase.
- Lab reports, in PDF format, must be submitted no later than 11:59pm on the day of demo, and must comply with the lab demo. Otherwise, your lab report will not be accepted/marked. The marking TAs will check the date of the demo and the date the lab report was submitted. Please submit only **one lab report** per group in onQ.
- Do not submit your lab report before the lab demo.
- Lab reports without a lab demo will not be accepted/marked.
- Consult the lab demo, report submission, and marking guidelines for each phase for additional information.

To support the needs of all learners, there are two "expected demo dates" for each phase. Groups can have their simulation/board demo in either of the two dates (e.g., Feb 2 or Feb 9 for Phase 1 for Monday Section), whenever they are ready, and then submit their lab report by 11:59pm on that same day. Also, as a universal accommodation, an automatic one-week extension has been considered with no penalty. Afterwards, late penalty will apply as specified in onQ.